Midterm Project Report

Name: Nguyễn Đình Khải
Student ID: 22070920
Email: 22070920@vnu.edu.vn

October 21, 2026

Abstract

This project implements and compares classical, learning-free computer vision techniques for three tasks. In Part A, Mean, Gaussian and Median filters and Laplacian sharpening are evaluated on two of our own photographs corrupted by Gaussian and salt-and-pepper noise, with 3×3, 5×5 and 7×7 kernels: averaged over both photos, a 3×3 median filter is best for salt-and-pepper noise (27.1 dB PSNR), and a 3×3 Gaussian filter with $\sigma=1$ for Gaussian noise (26.0 dB), although the best σ depends on the image content. In Part B, the fundamental matrix of the Middlebury *Venus* stereo pair is estimated with RANSAC (0.17 px Sampson error on inliers), and Block Matching (BM) is compared with Semi-Global Block Matching (SGBM) over a grid of parameters: SGBM gives denser and more accurate disparity maps (91.7% vs. 84.6% valid pixels) at about four times the cost, and its disparity is reprojected into a 3D point cloud. In Part C, SIFT and ORB are compared on four of our own photographs of a Hanoi street, with RANSAC homographies and feathering: SIFT finds twice as many matches, both reach about 1.5 px inlier RMSE, ORB is nine times faster to detect, and the panorama exposes real ghosting and perspective distortion. As a bonus, Laplacian-pyramid (multi-band) blending with hard-seam masks replaces the semi-transparent ghost of feathering by a single opaque copy on our panorama, and its overlap-sharpness index reaches 99.8% of the hard seam, against 86.3% for feathering.

1 Introduction

Classical computer vision relies on explicit mathematical models: convolution, projective geometry and epipolar geometry. These methods are interpretable, need no training data and run in real time on a CPU, so they remain the backbone of many practical systems, such as denoising and panorama modes in phone cameras, stereo depth sensing for robots and vehicles, and 3D mapping.

The goal of this project is to implement, evaluate and compare classical techniques for three tasks:

- **Part A – Image filtering:** removing noise and sharpening details, the first step of almost every imaging pipeline (camera, medical and surveillance images).
- **Part B – 3D reconstruction:** recovering the epipolar geometry, a dense disparity map and a 3D point cloud from two views, as used for depth estimation in robotics and autonomous driving.
- **Part C – Image stitching:** merging four overlapping photos into a panorama, as in phone panoramas and aerial mosaics.

Each part compares at least two methods or parameter settings with quantitative metrics (PSNR/SSIM, valid pixels, inlier ratio, RMSE, runtime) and visual inspection. Sections 2–4 present, for each part, the methodology, the implementation and results, and the comparative analysis; Section 5 concludes.

1.1 Experimental Setup

Environment. Python 3.12 with OpenCV 4.13, NumPy, scikit-image 0.26 (PSNR/SSIM), pandas and Matplotlib, on a Windows 11 laptop CPU. Random steps use a fixed seed (`seed=42`, `cv2.setRNGSeed(42)`), so all quality metrics are reproduced exactly. All tables and figures of this report come from one run of the accompanying notebook `22070920_NguyenDinhKhai.ipynb`. Runtimes are single measurements after a warm-up call; they vary by roughly ±30% between runs, so only relative differences should be interpreted.

Data.

- **Part A:** two of our own phone photographs. *desk*: a desk (keyboard, notebook, laptop, mouse) in daylight, cropped to remove a laptop screen and rotated to landscape, downscaled to 1024×635; it combines high-contrast edges and fine text (white letters on black keys, the notebook cover) with large flat regions (the wooden desk). *setup*: a computer setup (laptop, two monitors, backlit keyboard, mouse, window grille and medals behind), 2568×1926 downscaled to 1024×768; it has even finer detail (small text on the screens, key legends, thin window bars) next to smooth dark screens and the wooden desk. Each photo is the clean reference; noisy versions are synthesised from it, so PSNR and SSIM are measured against a true clean image. The tables report the mean over the two photos.
- **Part B:** the *Venus* pair of the Middlebury optical-flow benchmark [2] (420×380) with ground-truth flow. The vertical flow is zero everywhere, so the pair is rectified and the ground-truth disparity of the left image is $d = -u$, within $[-7, 9.4]$ px.
- **Part C:** four of our own photographs of a street in Hanoi (office and apartment buildings, shop fronts, trees, parked and moving vehicles), taken in sequence from left to right from the same spot by rotating a phone (main $1\times$ camera, 2000×1500). Each shot is rotated by about 20–25° and the lens covers about 75°, so the four photos span roughly 140°. Projected onto a single plane, the outer images are then stretched towards infinity: our first attempt on the full photos produced a 12752×5055 canvas with SIFT and 22392×8068 with ORB, which the program rejects. We therefore keep the central 50% of the width of each photo (1000×1500), equivalent to a narrower lens; neighbouring crops still overlap by about a third. The original photos are kept in `data/part_c/original/`.

2 Part A: Image Filtering

2.1 Methodology

Noise models. Additive Gaussian noise: $I_n = \text{clip}(I+\eta)$ with $\eta \sim \mathcal{N}(0, \sigma_n^2)$ and $\sigma_n = 20$ (on the 0–255 scale). Salt-and-pepper noise: each pixel is set to 0 with probability $p/2$ and to 255 with probability $p/2$, with $p = 0.05$.

Mean (box) filter replaces each pixel by the average of its $k \times k$ window W_k:

$$\hat{I}(x, y) = \frac{1}{k^2} \sum_{(i,j)\in W_k} I(x + i,\, y + j). \tag{1}$$

Gaussian filter weights the neighbours by their distance to the centre:

$$G_\sigma(i,j) = \frac{1}{2\pi\sigma^2}\exp\left(-\frac{i^2+j^2}{2\sigma^2}\right), \qquad \hat{I} = G_\sigma * I. \tag{2}$$

The kernel is separable into two 1D filters; σ, not k, controls the amount of smoothing as long as $k \gtrsim 6\sigma$.

Median filter is non-linear: $\hat{I}(x,y) = \text{median}\{I(x+i, y+j) : (i,j) \in W_k\}$. Impulse values (0 or 255) are almost never the median of a window, so they are removed instead of being spread out, and step edges are preserved.

Laplacian sharpening subtracts the second derivative to boost edges:

$$I_{\text{sharp}} = I - \alpha\,\nabla^2 I, \qquad \nabla^2 I = \frac{\partial^2 I}{\partial x^2} + \frac{\partial^2 I}{\partial y^2}. \tag{3}$$

$\nabla^2 I$ is computed with OpenCV's $k \times k$ Laplacian kernels ($k = 3, 5, 7$). Larger kernels have a much larger response (about $4\times$, $64\times$ and $1024\times$ the true second derivative), so each kernel is divided by its response to $f(x,y) = x^2$, whose true Laplacian is 2; the same α then has the same meaning for every k. The computation is done in floating point, keeping the sign of the Laplacian, before clipping to $[0, 255]$.

Metrics. MSE $= \frac{1}{N}\sum(I - \hat{I})^2$, PSNR $= 10\log_{10}(255^2/\text{MSE})$ and

$$\text{SSIM}(x,y) = \frac{(2\mu_x\mu_y + C_1)(2\sigma_{xy} + C_2)}{(\mu_x^2 + \mu_y^2 + C_1)(\sigma_x^2 + \sigma_y^2 + C_2)}\ [8], \tag{4}$$

all computed against the clean image, together with the runtime of each filter.

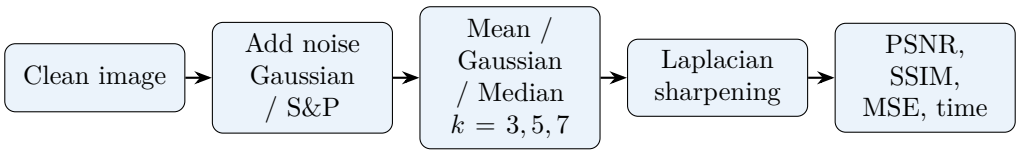

Figure 1: Experimental pipeline of Part A.

2.2 Implementation and Results

The filters use `cv2.blur`, `cv2.GaussianBlur` ($\sigma \in \{0.5, 1, 2\}$), `cv2.medianBlur` and `cv2.Laplacian` ($\alpha \in \{0.5, 1\}$), each with $k \in \{3, 5, 7\}$, i.e. 21 settings, all applied to both noise types. Figure 2 shows, for each noise type, the original, the noisy image, the best setting of each denoising filter, and the best denoised image after Laplacian sharpening (3×3, $\alpha = 0.5$).

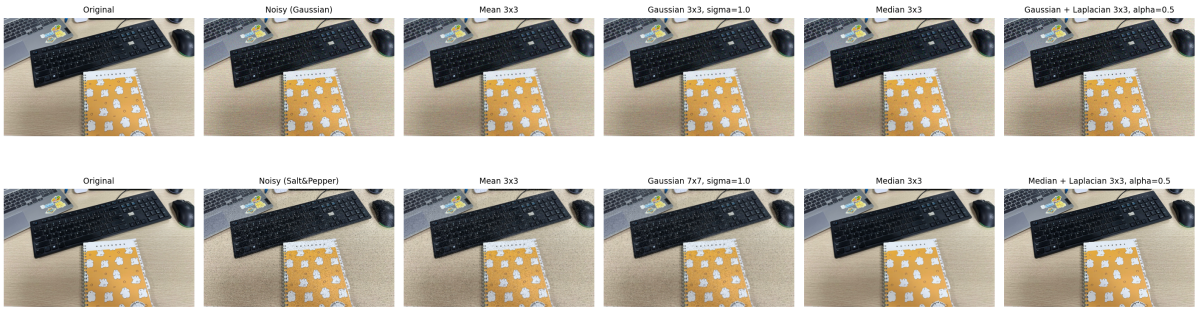

Figure 2: *desk*: original, noisy, denoised (best setting of each filter) and denoised-then-sharpened images. Top row: Gaussian noise; bottom row: salt-and-pepper noise.

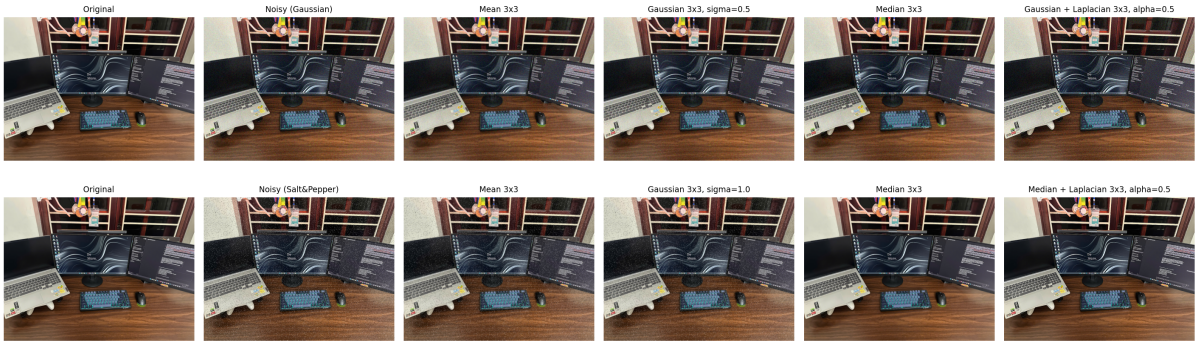

Figure 3: *setup*: same layout as Figure 2.

2.3 Comparative Analysis

Quantitative. Tables 1 and 2 list the metrics of every setting, averaged over the two photos (bold = best), Table 3 gives the best setting of each denoising filter on each photo, and Figure 4 plots PSNR against the kernel size.

Table 1: Part A – Gaussian noise ($\sigma_n = 20$), mean over the two photos.

filter	params	PSNR (dB)	SSIM	MSE	Time (ms)
None (noisy)	-	22.39	0.436	374.7	0.00
Mean	3x3	25.09	0.662	208.4	2.31
	5x5	23.02	0.620	346.2	3.19
	7x7	21.97	0.574	444.2	2.13
Gaussian	3x3, sigma=0.5	25.61	0.569	178.6	0.89
	3x3, sigma=1.0	**26.02**	0.690	**166.9**	1.31
	3x3, sigma=2.0	25.31	0.671	198.0	0.62
	5x5, sigma=0.5	25.61	0.569	178.6	0.92
	5x5, sigma=1.0	25.55	0.711	188.4	0.85
	5x5, sigma=2.0	23.73	0.663	292.6	0.81
	7x7, sigma=0.5	25.61	0.569	178.6	1.67
	7x7, sigma=1.0	25.48	**0.712**	191.8	1.18
	7x7, sigma=2.0	23.12	0.650	338.2	0.93
Median	3x3	25.22	0.613	197.6	1.18
	5x5	23.36	0.609	316.2	2.97
	7x7	22.25	0.584	417.0	74.30
Laplacian sharpen	3x3, alpha=0.5	16.43	0.276	1480.0	16.58
	3x3, alpha=1.0	13.34	0.208	3016.7	20.56
	5x5, alpha=0.5	20.76	0.400	546.4	20.86
	5x5, alpha=1.0	19.17	0.363	787.7	24.95
	7x7, alpha=0.5	21.68	0.425	442.1	22.03
	7x7, alpha=1.0	20.81	0.408	539.5	21.88

Table 2: Part A – salt-and-pepper noise ($p = 5\%$), mean over the two photos.

filter	params	PSNR (dB)	SSIM	MSE	Time (ms)
None (noisy)	-	17.80	0.400	1079.9	0.00
Mean	3x3	23.42	0.544	301.9	2.18
	5x5	22.45	0.547	390.5	2.86
	7x7	21.65	0.534	474.7	2.38
Gaussian	3x3, sigma=0.5	21.30	0.484	482.9	0.68
	3x3, sigma=1.0	23.86	0.567	271.4	0.68
	3x3, sigma=2.0	23.56	0.552	292.1	0.59
	5x5, sigma=0.5	21.30	0.484	482.9	0.83
	5x5, sigma=1.0	24.02	0.599	264.3	1.03
	5x5, sigma=2.0	23.02	0.584	340.1	1.05
	7x7, sigma=0.5	21.30	0.484	482.9	1.18
	7x7, sigma=1.0	24.01	0.601	265.3	0.92
	7x7, sigma=2.0	22.64	0.597	374.7	0.92
Median	3x3	**27.07**	**0.855**	**131.3**	1.05
	5x5	23.72	0.721	293.5	3.32
	7x7	22.28	0.648	416.2	63.39
Laplacian sharpen	3x3, alpha=0.5	16.63	0.366	1415.0	16.69
	3x3, alpha=1.0	15.30	0.320	1921.0	16.21
	5x5, alpha=0.5	17.55	0.400	1144.1	23.17
	5x5, alpha=1.0	17.00	0.380	1299.1	22.06
	7x7, alpha=0.5	17.70	0.404	1104.2	22.44
	7x7, alpha=1.0	17.45	0.397	1169.4	22.53

Table 3: Part A – best setting (highest PSNR) of each denoising filter on each photo.

noise	image	filter	params	PSNR (dB)	SSIM
Gaussian	desk	Mean	3x3	26.26	0.652
	desk	Gaussian	3x3, sigma=1.0	27.03	0.687
	desk	Median	3x3	25.88	0.600
Salt&Pepper	desk	Mean	3x3	24.32	0.541
	desk	Gaussian	7x7, sigma=1.0	25.01	0.600
	desk	Median	3x3	28.12	0.823
Gaussian	setup	Mean	3x3	23.93	0.672
	setup	Gaussian	3x3, sigma=0.5	25.49	0.561
	setup	Median	3x3	24.56	0.626
Salt&Pepper	setup	Mean	3x3	22.53	0.547
	setup	Gaussian	3x3, sigma=1.0	23.12	0.566
	setup	Median	3x3	26.03	0.887

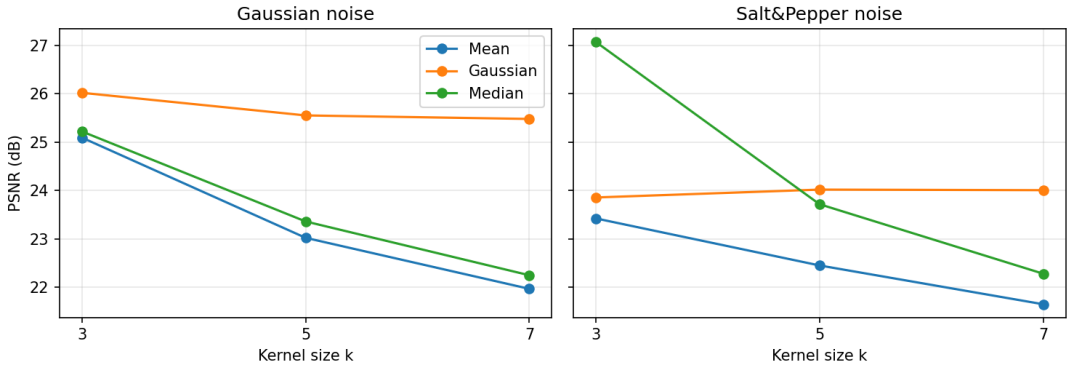

Figure 4: PSNR versus kernel size for the denoising filters (Gaussian with $\sigma = 1$).

Observations.

- **Gaussian noise:** on average the Gaussian filter with $\sigma = 1$ is best: 26.0 dB with $k = 3$, while $k = 5, 7$ give a slightly lower PSNR (25.6 and 25.5 dB) but the highest SSIM (0.71). It is ahead of the 3×3 median (25.2 dB) and mean (25.1 dB) filters. Gaussian noise has small amplitude and affects every pixel, so a weighted average is close to optimal; the median, designed to reject outliers, brings no advantage.
- **The best parameter depends on the image** (Table 3): on *desk*, $\sigma = 1$ wins clearly (27.0 dB against 25.7 dB for $\sigma = 0.5$), but on the more detailed *setup* photo the lighter $\sigma = 0.5$ gives the highest PSNR (25.5 vs. 25.0 dB), because stronger smoothing erases the small screen text and key legends. SSIM still prefers $\sigma = 1$ on *setup* (0.69 vs. 0.56): PSNR rewards keeping fine detail, SSIM rewards removing the noise in flat regions, so the two metrics can disagree and the filter should be tuned to the content and to the target metric.
- **Salt-and-pepper noise:** the 3×3 median filter is far ahead on both photos (27.1 dB and SSIM 0.86 on average, from 17.8 dB for the noisy images; 28.1 dB on *desk*, 26.0 dB on *setup*). Mean and Gaussian filters reach at most 24.0 dB: they *spread* each 0/255 impulse into a grey blob instead of removing it (Figures 2 and 3, bottom rows).

- **Noise removal vs. edge preservation (kernel size):** larger kernels remove more noise but blur edges (over-smoothing). On these sharp photos with many thin edges (key legends, screen text, window bars) the effect is strong: the 7×7 median drops to 22.3 dB on salt-and-pepper noise, and on Gaussian noise the 7×7 mean falls to 22.0 dB against 25.1 dB for 3×3; Gaussian $\sigma = 2$ with $k = 7$ also loses about 2.4 dB with respect to $\sigma = 1$. The Gaussian filter with $\sigma = 0.5$ gives *identical* results for every k, because the weights outside the central 3×3 are almost zero: σ sets the smoothing, and k only needs to be about 6σ.
- **Laplacian sharpening and halos:** applied to a noisy image, sharpening lowers PSNR to 13–22 dB, because the second derivative amplifies high-frequency noise; $\alpha = 1$ is worse than $\alpha = 0.5$, and the larger, smoother 7×7 kernel degrades least. After denoising (last columns of Figures 2 and 3) it makes edges crisper, but it also creates bright/dark *halos* (overshoot) along strong edges and brings back some residual noise, so it should follow denoising and use a small α.
- **Runtime** (mean over the two 1024-pixel-wide photos): Gaussian (separable, about 0.6–2 ms) and mean (running sums, about 2–3.7 ms) filters are almost independent of k. The median filter goes from about 1 ms (3×3) and 3–4 ms (5×5) to about 70 ms for 7×7, because OpenCV switches to a histogram-based algorithm for $k > 5$; Laplacian sharpening costs 16–26 ms because of the floating-point arithmetic.

Table 4: Part A – strengths and weaknesses of each filter.

Filter	Strengths	Weaknesses	Best for
Mean	simple, low computational cost, nearly independent of k	blurs edges, smears impulses	mild Gaussian noise
Gaussian	smooth weighting, separable, tunable σ	still blurs edges at large σ	Gaussian noise
Median	removes impulses, keeps step edges	slower for $k > 5$, erodes thin lines and corners	salt-and-pepper noise
Laplacian	enhances edges and fine detail	amplifies noise, halos for large α	after denoising, small α

3 Part B: 3D Reconstruction

3.1 Methodology

Epipolar geometry. For corresponding points $\mathbf{x}$ (left) and $\mathbf{x}'$ (right) in homogeneous coordinates, the rank-2 fundamental matrix F satisfies

$$\mathbf{x}'^{\top} F\, \mathbf{x} = 0, \qquad \mathbf{l}' = F\mathbf{x}, \quad \mathbf{l} = F^{\top}\mathbf{x}', \tag{5}$$

where $\mathbf{l}'$ is the epipolar line of $\mathbf{x}$ in the right image [4]. F is estimated using OpenCV RANSAC: repeatedly sample a minimal subset of matches, fit F, count the matches closer than a threshold t to their epipolar lines, keep the best model and refit it on all its inliers. Its accuracy is measured by the symmetric point-to-epipolar-line distance and

the **Sampson error**:

$$d_{\text{epi}} = \tfrac{1}{2}\left(\frac{|\mathbf{x}'^{\top} F \mathbf{x}|}{\|(F\mathbf{x})_{1:2}\|} + \frac{|\mathbf{x}'^{\top} F \mathbf{x}|}{\|(F^{\top}\mathbf{x}')_{1:2}\|}\right), \qquad d_{\text{S}}^2 = \frac{(\mathbf{x}'^{\top} F \mathbf{x})^2}{(F\mathbf{x})_1^2 + (F\mathbf{x})_2^2 + (F^{\top}\mathbf{x}')_1^2 + (F^{\top}\mathbf{x}')_2^2}. \quad (6)$$

Disparity. In a rectified pair, pixel (x, y) of the left image corresponds to $(x - d, y)$ in the right image. **Block Matching (BM)** picks the d that minimises the sum of absolute differences over a $b \times b$ window (local winner-takes-all). **SGBM** [3] minimises the global energy

$$E(D) = \sum_{\mathbf{p}} C(\mathbf{p}, D_{\mathbf{p}}) + \sum_{\mathbf{q} \in N_{\mathbf{p}}} P_1 \left[|D_{\mathbf{p}} - D_{\mathbf{q}}| = 1\right] + \sum_{\mathbf{q} \in N_{\mathbf{p}}} P_2 \left[|D_{\mathbf{p}} - D_{\mathbf{q}}| > 1\right], \quad (7)$$

approximately, by aggregating matching costs along several 1D paths: $L_r(\mathbf{p}, d) = C(\mathbf{p}, d) + \min\{L_r(\mathbf{p} - \mathbf{r}, d), L_r(\mathbf{p} - \mathbf{r}, d \pm 1) + P_1, \min_k L_r(\mathbf{p} - \mathbf{r}, k) + P_2\} - \min_k L_r(\mathbf{p} - \mathbf{r}, k)$. We use the usual penalties $P_1 = 8 \cdot 3 \cdot b^2$ and $P_2 = 32 \cdot 3 \cdot b^2$: P_1 allows slanted surfaces, P_2 preserves depth discontinuities.

3D reconstruction. From the disparity, `cv2.reprojectImageTo3D` computes $[X\, Y\, Z\, W]^{\top} = Q[x\, y\, d\, 1]^{\top}$:

$$Q = \begin{bmatrix} 1 & 0 & 0 & -c_x \\ 0 & 1 & 0 & -c_y \\ 0 & 0 & 0 & f \\ 0 & 0 & 1/B & \Delta/B \end{bmatrix} \Rightarrow\ Z = \frac{fB}{d + \Delta}, \quad X = \frac{(x - c_x)Z}{f}, \quad Y = \frac{(y - c_y)Z}{f}. \quad (8)$$

No calibration is supplied with our input files, so $f = 420$ px, the left principal point at the image centre and $B = 1$ are assumed. We define $\Delta = c'_x - c_x = 12$ px as an assumed disparity offset, giving $Z = fB/(d + \Delta)$ for $d = x_L - x_R$. This produces an illustrative point cloud; its shape and metric dimensions are not guaranteed without calibration.

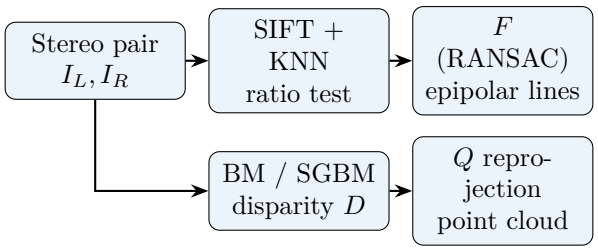

Figure 5: Pipeline of Part B.

3.2 Implementation and Results

Fundamental matrix. SIFT keypoints are matched with `BFMatcher.knnMatch` ($k = 2$) and Lowe's ratio test (0.75), and F is estimated with `cv2.findFundamentalMat(..., cv2.FM_RANSAC, 1.0, 0.999)`. Figure 6 draws 15 epipolar lines on *both* images: in each image, the lines of the matching points of the other image pass through the corresponding points.

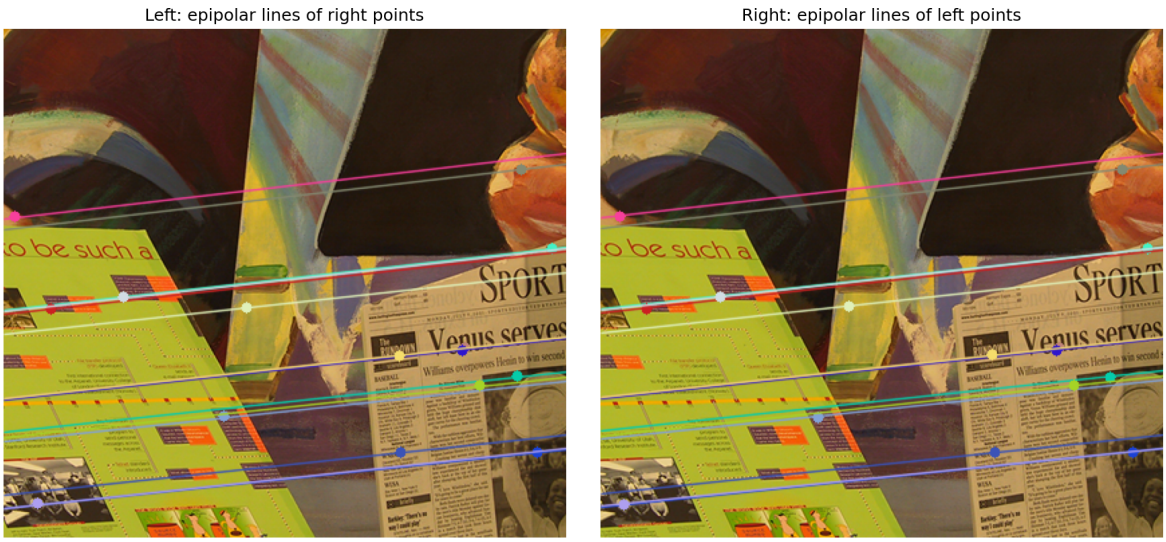

Figure 6: Epipolar lines (SIFT + RANSAC, $t = 1$ px). Each colour is one match and its epipolar line.

Disparity. `cv2.StereoBM` (on grey images) and `cv2.StereoSGBM` (on colour images) are run over `blockSize` $\in \{5, 9, 15, 21\}$ $\times$ `numDisparities` $\in \{16, 32, 48, 64\}$, i.e. 32 configurations, with `minDisparity`$= -16$, `uniquenessRatio`$= 10$ and a speckle filter (window 100, range 2). Each map is compared with the ground truth. Figure 7 shows the best configuration of each method.

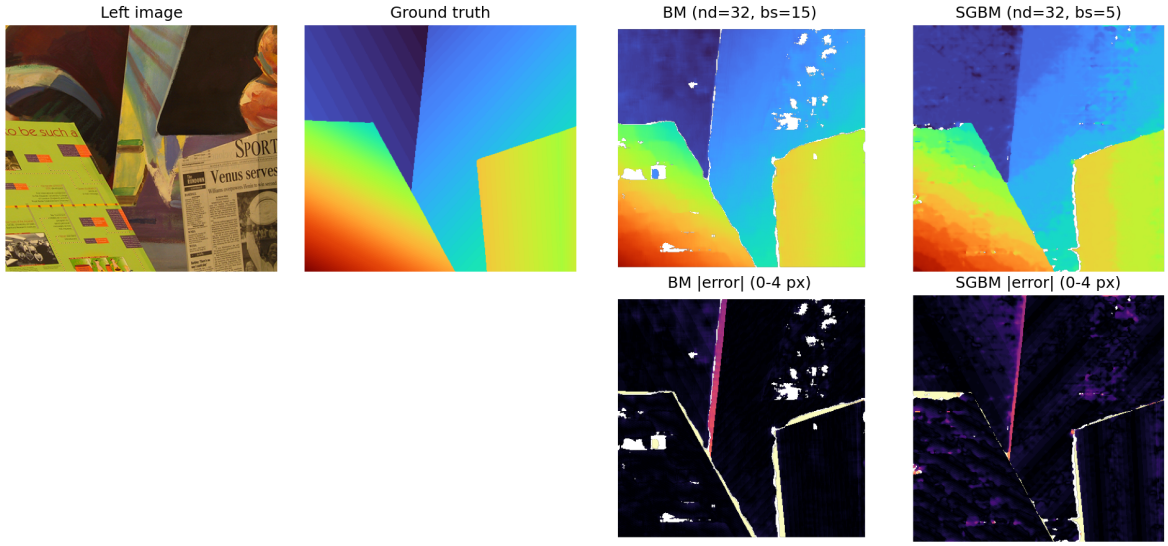

Figure 7: Colour-coded disparity (same colour scale as the ground truth; white = invalid) and absolute error (0–4 px) of the best BM and SGBM configurations.

Point cloud. The best SGBM disparity is reprojected with Q; the 1% nearest and farthest points are discarded as outliers, leaving about 143 000 coloured points, saved as `pointcloud_sgbm.ply` and shown from four viewpoints in Figure 8.

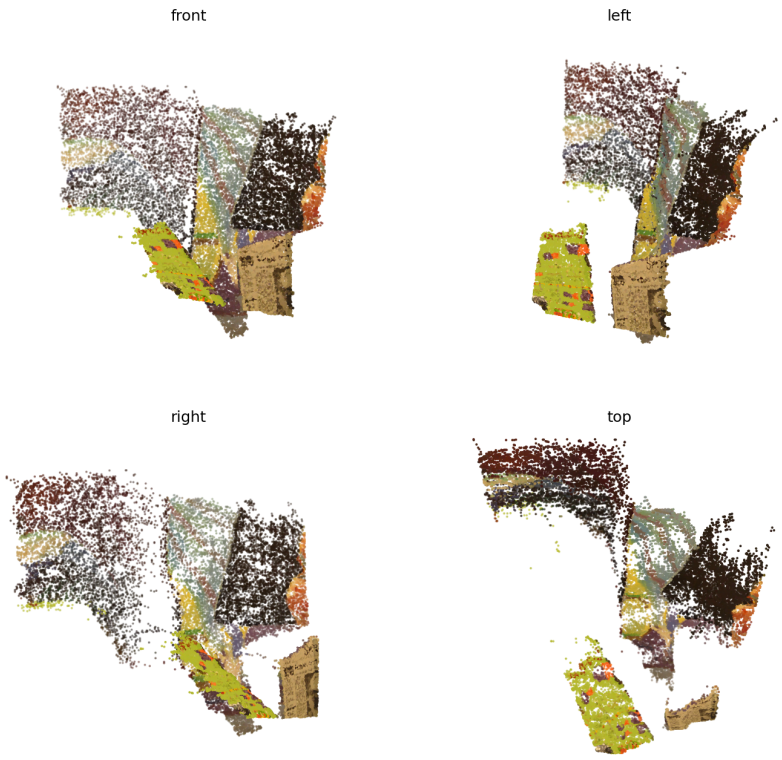

Figure 8: Point cloud reconstructed from the SGBM disparity, seen from the front, left, right and top.

3.3 Comparative Analysis

Fundamental matrix (Table 5). Of 411 ratio-test matches, RANSAC keeps 381 inliers (92.7%), with a mean epipolar distance of 0.23 px and a Sampson error of 0.17 px on the inliers; over all matches the Sampson error is 1.7 px, which shows how strongly the remaining outliers would bias a non-robust estimate. In Figure 6 the lines are almost horizontal, as expected for a rectified pair, and pass through the matching points in both images.

Table 5: Fundamental-matrix estimation with RANSAC (SIFT matches).

metric	**value**
keypoints (L/R)	629/567
matches	411
inliers	381
inlier ratio (%)	92.701
epipolar dist inliers (px)	0.233
Sampson inliers (px)	0.165
Sampson all (px)	1.705
F time (ms)	0.546

Table 6: BM vs. SGBM – best configuration of each method, selected by "bad>2 + inv.", the percentage of pixels that are invalid or wrong by more than 2 px.

method	numDisp.	blockSize	valid (%)	RMSE (px)	bad>1 (%)	bad>2 (%)	bad>2 + inv. (%)	runtime (ms)
BM	32	15	84.6	0.85	3.72	2.96	17.88	**6.4**
SGBM	32	5	**91.7**	**0.69**	2.84	1.74	**9.91**	26.1

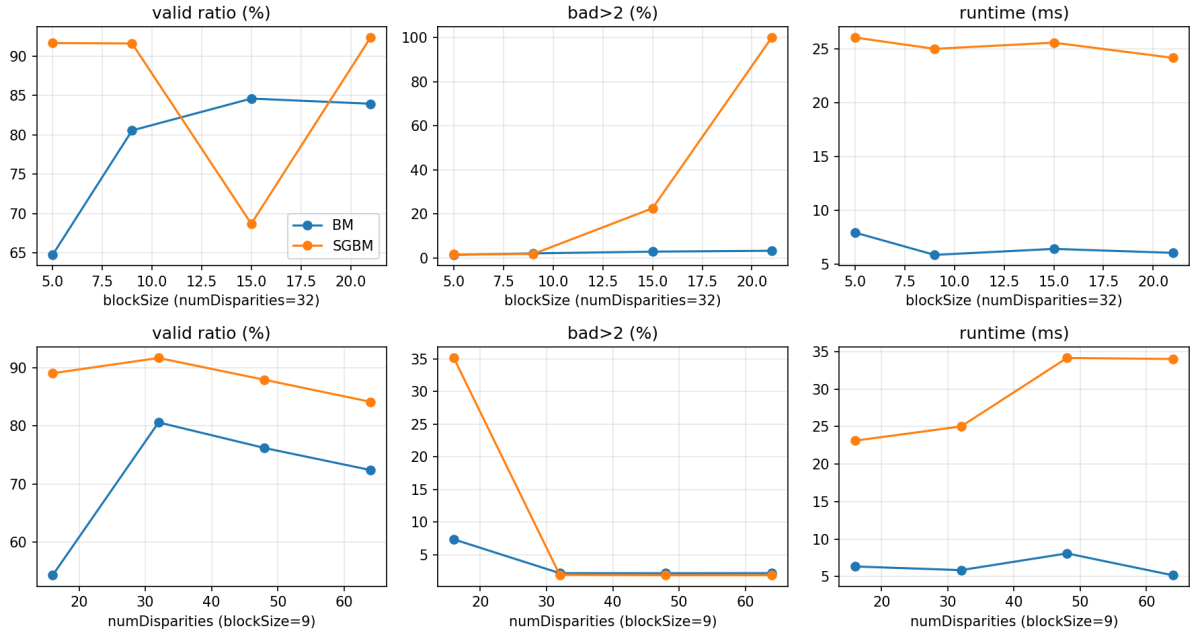

Figure 9: Valid ratio, bad-pixel rate and runtime versus `blockSize` (top, `numDisparities`=32) and versus `numDisparities` (bottom, `blockSize`=9).

BM vs. SGBM (Tables 6 and 7, Figure 9).

- **Valid disparity ratio:** the best SGBM map has 91.7% valid pixels against 84.6% for the best BM map. BM leaves holes in low-texture areas (the dark poster at the top right) and around depth discontinuities, where the window covers two surfaces. SGBM propagates information through the smoothness terms P_1, P_2 and fills these regions.
- **Accuracy:** SGBM is also more accurate (RMSE 0.69 vs. 0.85 px, 1.7% vs. 3.0% of valid pixels wrong by more than 2 px). Counting invalid pixels as errors, 9.9% of SGBM pixels are bad against 17.9% for BM.
- **Smoothness and object boundaries:** SGBM surfaces are smooth and its errors concentrate on occlusion boundaries (Figure 7, bottom row). BM with a 15×15 window shows thicker error bands along depth edges ("foreground fattening": objects grow by up to half a window).
- **`blockSize`:** for BM, larger windows increase density (64.7% at $b = 5$ to 84.6% at $b = 15$) but also blur boundaries (bad>1 from 1.9% to 4.4% at $b = 21$). For SGBM, $b \geq 15$ *breaks down* (22.6% bad pixels at $b = 15$, and every pixel wrong at $b = 21$): the penalties grow with b^2 ($P_2 = 32 \cdot 3 \cdot b^2 = 21\,600$ at $b = 15$ and 42 336 at $b = 21$, above the 16-bit signed limit of 32 767). Our hypothesis is that the aggregated path

costs then overflow the 16-bit cost type of OpenCV's implementation; we did not verify this inside the library. The OpenCV documentation [1] gives 3–11 as the usual range of `blockSize`; in our experiments, with penalties proportional to b^2, SGBM stays correct up to $b = 9$. These configurations are kept as documented failure cases.

- **`numDisparities`:** with 16 disparities the search range is only $[-16, -1]$ and cannot reach the positive disparities of Venus: BM rejects about half of the pixels (54% valid) and SGBM assigns wrong values on a large scale (35% bad pixels). Beyond the needed range ($nd > 32$) the invalid band on the left grows (SGBM: 91.6% $\rightarrow$ 84.1% valid from 32 to 64) and SGBM becomes slower (25 $\rightarrow$ 34 ms), because its cost grows linearly with the number of disparities.
- **Runtime:** BM is the fastest (about 5–9 ms) and almost independent of the parameters; SGBM costs about 21–35 ms, roughly four to five times more, which is the price of its global smoothness constraint.

3D geometry. The point cloud reproduces the slanted planes of the Venus scene (Figure 8): the posters appear as separate tilted planes at different depths. Holes appear where SGBM had no valid disparity, and "flying pixels" bridge foreground and background along depth discontinuities, where the disparity is interpolated across the edge. Since $Z \propto 1/(d + \Delta)$, a small disparity error on a distant surface produces a much larger depth error than on a near one. With the assumed camera and offset, this cloud is illustrative; calibration is needed to assess its geometric accuracy.

Table 7: All BM and SGBM configurations.

method	numDisp.	blockSize	valid (%)	RMSE (px)	bad>1 (%)	bad>2 (%)	bad>2 + inv. (%)	runtime (ms)
BM	16	5	42.2	1.21	3.42	2.69	58.93	6.4
	16	9	54.3	2.85	8.59	7.36	49.67	6.3
	16	15	58.9	3.56	12.77	10.95	47.56	5.0
	16	21	58.4	3.65	12.74	10.40	47.70	4.9
	32	5	64.7	0.62	1.86	1.49	36.22	7.9
	32	9	80.6	0.72	2.69	2.18	21.19	5.9
	32	15	84.6	0.85	3.72	2.96	17.88	6.4
	32	21	84.0	0.91	4.40	3.36	18.85	6.1
	48	5	60.3	0.61	1.87	1.48	40.63	8.6
	48	9	76.2	0.72	2.71	2.17	25.48	8.1
	48	15	80.7	0.84	3.75	2.98	21.74	6.4
	48	21	80.3	0.91	4.46	3.39	22.41	7.7
	64	5	56.5	0.61	1.93	1.51	44.40	6.0
	64	9	72.4	0.71	2.75	2.19	29.19	5.2
	64	15	76.9	0.84	3.77	2.96	25.39	5.4
	64	21	76.8	0.92	4.58	3.46	25.86	5.0
SGBM	16	5	88.2	4.98	37.22	34.76	42.45	22.1
	16	9	89.0	4.64	37.63	35.14	42.27	23.1
	16	15	62.2	8.27	29.59	27.15	54.68	22.6
	16	21	96.2	15.29	100.00	100.00	100.00	21.0
	32	5	91.7	0.69	2.84	1.74	9.91	26.1
	32	9	91.6	0.72	2.71	1.90	10.12	25.0
	32	15	68.7	6.81	23.59	22.62	46.84	25.6
	32	21	92.4	15.19	100.00	100.00	100.00	24.2
	48	5	87.9	0.66	2.77	1.65	13.54	31.1
	48	9	87.9	0.70	2.68	1.85	13.74	34.1
	48	15	57.0	15.48	34.79	33.63	62.16	32.9
	48	21	88.6	15.10	100.00	100.00	100.00	29.5
	64	5	84.1	0.65	2.71	1.61	17.23	34.4
	64	9	84.1	0.69	2.69	1.85	17.45	34.0
	64	15	49.0	23.98	40.88	39.52	70.40	35.2
	64	21	84.8	15.01	100.00	100.00	100.00	34.1

4 Part C: Image Stitching

4.1 Methodology

Features. *SIFT* [5] detects extrema in a Difference-of-Gaussians scale space and describes each keypoint by 128-D histograms of gradient orientations (4×4 cells $\times$ 8 bins), making it invariant to scale and rotation. *ORB* [6] combines FAST corners, an orientation from the intensity centroid, and 256-bit rotated BRIEF binary descriptors; it is much faster but less precise.

Matching. Each descriptor of image i is matched to its two nearest neighbours in image $i+1$ ($k = 2$; Euclidean L2 distance for SIFT, Hamming distance for the binary ORB descriptors) and kept only if $d_1 < 0.75\, d_2$ (Lowe's ratio test), which rejects ambiguous matches.

Homography. Two views of a planar scene, or views under pure camera rotation, are related by $\mathbf{x}' \sim H\mathbf{x}$, where H is a 3×3 matrix with 8 degrees of freedom (defined up to scale). For pure rotation with the same camera intrinsics, $H = KRK^{-1}$; a planar scene with translation also requires a plane-dependent term. Each correspondence gives two linear equations, so 4 matches determine H by the Direct Linear Transform. RANSAC repeatedly samples 4 matches, fits H and counts the inliers with $\|\mathbf{x}' - \pi(H\mathbf{x})\| < t$, where

π divides by the third coordinate; it needs

$$N = \frac{\log(1-p)}{\log(1-w^4)} \tag{9}$$

iterations to draw an all-inlier sample with probability p when the inlier fraction is w. The best model is refined on all its inliers, and its accuracy is the inlier reprojection RMSE

$$\text{RMSE} = \sqrt{\frac{1}{|\mathcal{I}|}\sum_{i\in\mathcal{I}} \|\mathbf{x}'_i - \pi(H\mathbf{x}_i)\|^2}. \tag{10}$$

Warping and blending. The middle image is the reference frame, which shortens the homography chains and limits the stretching of the outer images. With $H_{i+1\to i}$ estimated for each adjacent pair, the images are chained to the reference r by $H_{i\to r} = H_{i-1\to r}H_{i\to i-1}$ on the right and $H_{i\to r} = H_{i+1\to r}H^{-1}_{i+1\to i}$ on the left. The canvas is the bounding box of the warped image corners, shifted by a translation T so that all coordinates are positive, and each image is warped with $TH_{i\to r}$. Overlaps are blended by *feathering*:

$$I(\mathbf{p}) = \frac{\sum_k w_k(\mathbf{p})\, I_k(\mathbf{p})}{\sum_k w_k(\mathbf{p})}, \tag{11}$$

where $w_k(\mathbf{p})$ is the distance from $\mathbf{p}$ to the border of image k (distance transform), so the weight of each image falls smoothly to zero at its border.

Multi-band (Laplacian pyramid) blending (bonus). Feathering averages the images over the whole overlap, so a moving object or a small misalignment becomes a semi-transparent ghost. Multi-band blending [7] blends each frequency band over a width proportional to its wavelength. First, a hard seam assigns every pixel to the image with the largest w_k, giving binary masks M_k. Each warped image is decomposed into a Laplacian pyramid $L_l(I_k) = G_l(I_k) - \text{up}(G_{l+1}(I_k))$, $l = 0, \ldots, n-1$, with $L_n = G_n$, where G_l is the Gaussian pyramid (blur and subsample by 2). Each band is blended with the mask blurred to the same level, and the panorama is rebuilt from the coarsest level:

$$L_l(I) = \frac{\sum_k G_l(M_k)\, L_l(I_k)}{\sum_k G_l(M_k)}, \qquad I = L_0 + \text{up}\big(L_1 + \text{up}(L_2 + \ldots)\big). \tag{12}$$

Low frequencies (exposure, sky colour) are thus mixed over about 2^n pixels and leave no step, while high frequencies (edges, texture) are mixed over only a few pixels around the seam, so details are not averaged across the whole overlap as in feathering. Before building the pyramids, the pixels outside each warped image are filled with the colour of the nearest valid pixel; otherwise the black background would leak into the coarse levels and darken the image borders. With $n = 0$ the method reduces to the hard seam without blending.

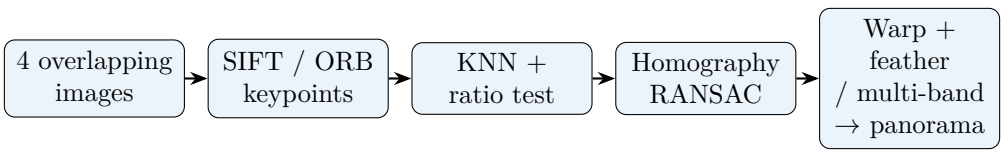

Figure 10: Pipeline of Part C.

4.2 Implementation and Results

Keypoints come from `cv2.SIFT_create` and `cv2.ORB_create` (5000 features) and are matched with `cv2.BFMatcher.knnMatch`. The homography of every adjacent pair is estimated with `cv2.findHomography(src, dst, cv2.RANSAC, t)` for $t \in \{1, 3, 5, 10\}$ px, and the panorama uses $t = 3$ px. Images are warped with `cv2.warpPerspective`; their footprint masks are eroded by one pixel to drop border pixels interpolated with the black background, and feathering weights come from `cv2.distanceTransform`. If a pair has fewer than 10 inliers, or the canvas would exceed 30 megapixels (homography chain stretched too far), the program reports it and skips that panorama instead of producing a wrong mosaic. The total stitching time includes detection, matching, RANSAC, warping and blending (feathering). The inputs are the four cropped street photos described in Section 1.1.

For the blending study (bonus), the SIFT panorama is also blended with a hard seam and with multi-band blending of $n \in \{2, 4, 6\}$ levels (`cv2.pyrDown`/`cv2.pyrUp`; the nearest-pixel filling uses `cv2.distanceTransformWithLabels`). Without ground truth, two measures are computed on the overlaps (pixels covered by at least two images): the *seam gradient*, the mean Sobel gradient magnitude on the hard-seam line (a visible seam is a step, so a lower value means a less visible seam), and the *overlap sharpness*, the mean $|\nabla^2 I|$ over the overlaps (averaging two misaligned images blurs them and lowers it), also given relative to the hard seam, which copies every pixel from a single image. Figure 16 enlarges the two 400×400 regions where the methods differ most.

Figure 11: Keypoints detected by SIFT (top) and ORB (bottom) on the four input images.

Figure 12: RANSAC inlier matches ($t = 3$ px) between images 2 and 3 (top: SIFT, bottom: ORB); at most about 150 lines are drawn.

Figure 13: Final panorama (SIFT + feathering, 5437×3331 canvas). Note the stretched left image, the smooth transitions in the sky, and the semi-transparent "ghost" motorcyclist in front of the white car on the right.

Figure 14: Panoramas obtained with SIFT (top) and ORB (bottom).

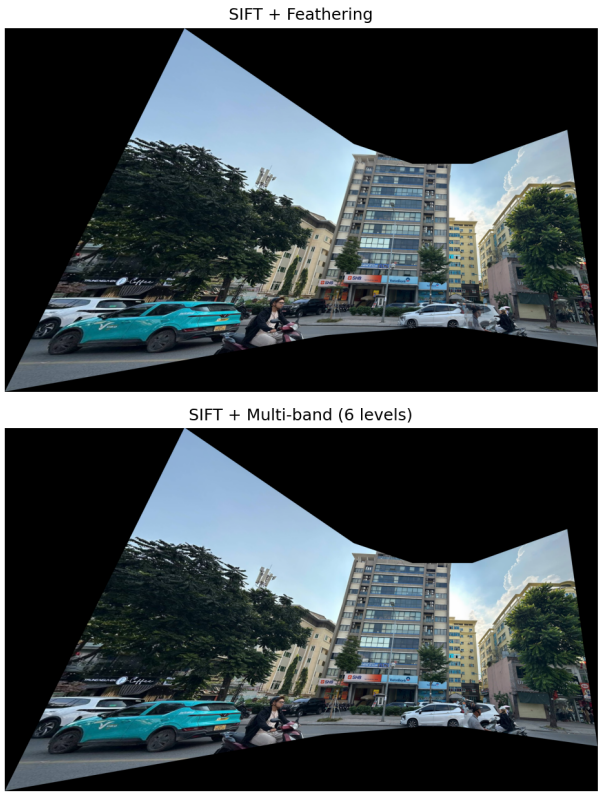

Figure 15: Bonus: SIFT panorama blended by feathering (top) and by multi-band blending with 6 pyramid levels (bottom).

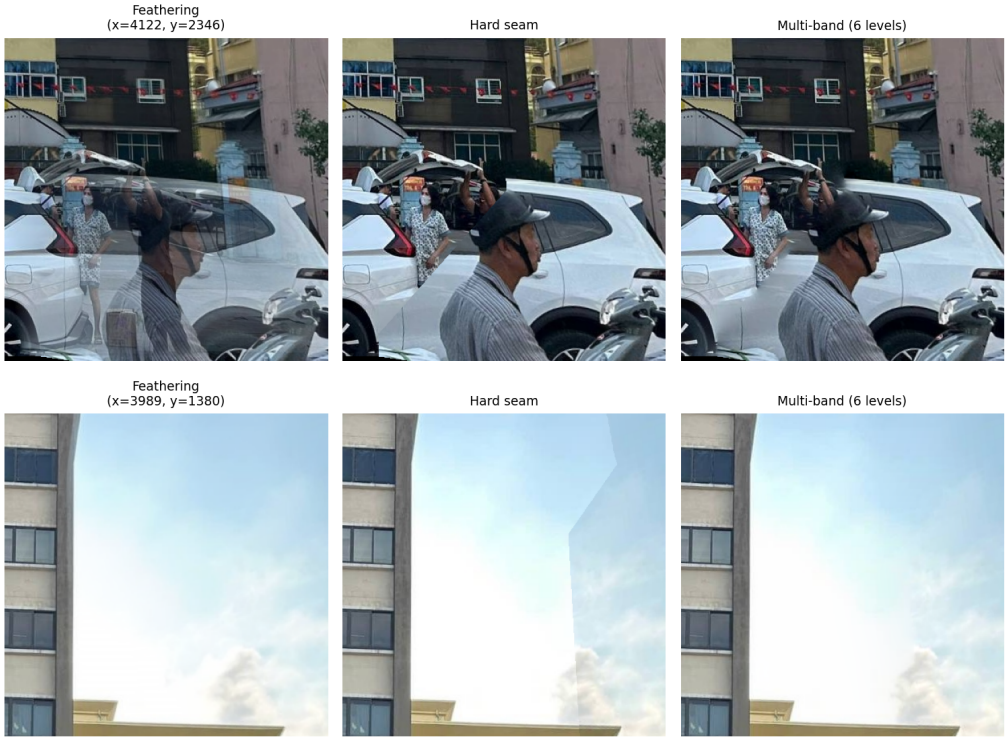

Figure 16: Bonus: enlarged 400×400 regions where the methods differ most (canvas coordinates in the first column). Top: a motorcyclist present in only one of the overlapping shots. Bottom: the sky next to the tall building, where the exposure changed between shots.

4.3 Comparative Analysis

Table 8: SIFT vs. ORB (totals over the three pairs, RANSAC $t = 3$ px; total = whole stitching time).

detector	keypoints / img	matches	inliers	inlier ratio (%)	RMSE (px)	detect (ms)	match (ms)	total (ms)
SIFT	8351	3213	1873	58.3	**1.481**	1059	832	4145
ORB	5000	1622	1079	**66.5**	1.493	119	341	**3355**

Table 9: Effect of the RANSAC threshold for each image pair.

detector	pair	RANSAC t (px)	matches	inliers	inlier ratio (%)	RMSE (px)
SIFT	1-2	1	1012	190	18.8	0.623
	1-2	3	1012	520	51.4	1.746
	1-2	5	1012	728	71.9	2.506
	1-2	10	1012	892	88.1	3.609
	2-3	1	1044	258	24.7	0.599
	2-3	3	1044	510	48.9	1.467
	2-3	5	1044	663	63.5	2.361
	2-3	10	1044	824	78.9	3.382
	3-4	1	1157	451	39.0	0.569
	3-4	3	1157	843	72.9	1.301
	3-4	5	1157	931	80.5	1.726
	3-4	10	1157	985	85.1	2.323
ORB	1-2	1	320	59	18.4	0.587
	1-2	3	320	161	50.3	1.581
	1-2	5	320	214	66.9	2.299
	1-2	10	320	281	87.8	3.094
	2-3	1	595	151	25.4	0.675
	2-3	3	595	339	57.0	1.503
	2-3	5	595	410	68.9	2.089
	2-3	10	595	469	78.8	3.343
	3-4	1	707	233	33.0	0.663
	3-4	3	707	579	81.9	1.462
	3-4	5	707	666	94.2	1.731
	3-4	10	707	668	94.5	1.766

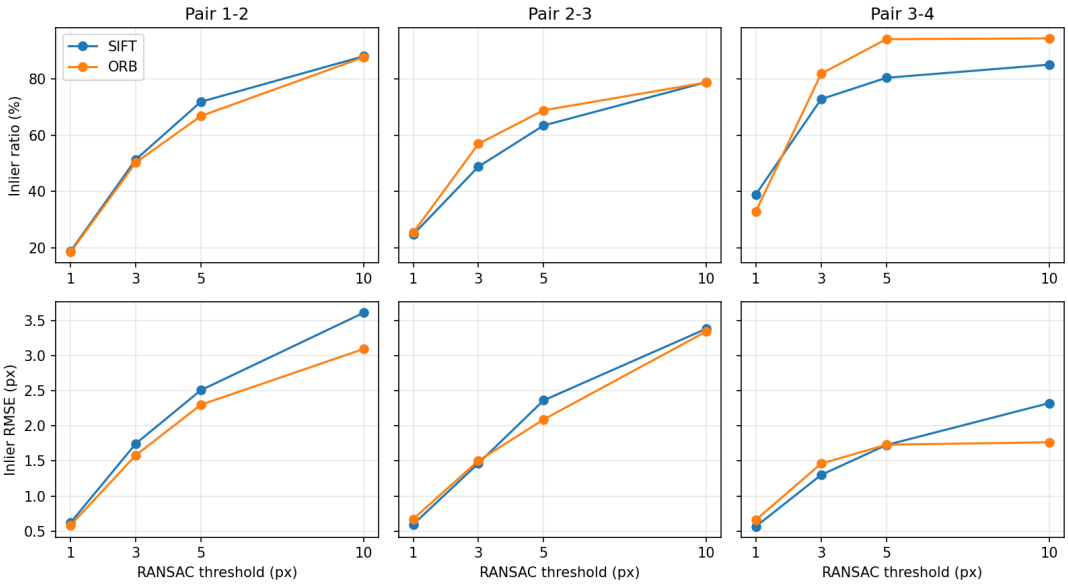

Figure 17: Inlier ratio (top) and inlier RMSE (bottom) versus the RANSAC threshold, for each image pair.

Quantitative.

- **Keypoints and matches:** on these large, textured photos SIFT detects about 8350 keypoints per image, while ORB stops at its 5000 limit; SIFT also yields twice as many ratio-test matches (3213 vs. 1622 over the three pairs) and more inliers (1873 vs. 1079). Inliers concentrate on the far, static building façades (Figure 12); few are found on the road, the moving vehicles and the trees.

- **Inlier ratio and accuracy:** at $t = 3$ px only 58% (SIFT) and 67% (ORB) of the matches are inliers, far below what an exact homography would give. A homography explains a pure camera rotation; here the phone also moved slightly while turning, so near objects (cars, trees, the pavement) shift differently from the far buildings (parallax), vehicles and people moved between shots, and the facades contain many repeated windows that produce wrong matches. Both detectors reach a similar inlier RMSE (1.48 and 1.49 px): with real photos the error is dominated by these effects rather than by keypoint localisation. ORB's higher ratio comes with half as many matches, clustered on a few high-contrast corners of one facade (air-conditioning units, window frames; Figure 12), whereas SIFT inliers are spread over the whole building.
- **RANSAC threshold** (Table 9, Figure 17): the threshold now matters a lot. At $t = 1$ px only 18–39% of the matches are kept, so many correct but slightly imperfect matches are rejected; at $t = 10$ px 79–94% are kept, but RMSE grows from about 0.6 px to 1.8–3.6 px, because points affected by parallax are accepted and the homography becomes a compromise between near and far surfaces. $t = 3$ px balances these effects (49–82% inliers, 1.3–1.8 px RMSE) and is used for the panorama.
- **Runtime:** SIFT detection is about nine times slower than ORB (about 1060 vs. 120 ms for four images) and its matching is also slower because it has more descriptors (about 830 vs. 340 ms). RANSAC takes less than 10 ms. Most of the time is spent on the large canvas: warping takes about 0.3 s and feathering 2–2.5 s, so the total stitching time is about 4.1 s with SIFT and 3.4 s with ORB.

Qualitative (Figures 13 and 14).

- **Seam visibility and illumination transitions:** feathering leaves no hard seam. The automatic exposure of the phone changed between shots (the sky gets brighter towards the right, where the camera faces the sun); the distance weights turn these differences into smooth gradients across the overlaps instead of visible steps, although no exposure compensation is applied.
- **Geometric alignment:** the static, distant structures are well aligned: the tall building, its windows and bank signs, the shop signs and the trees are continuous across the overlaps, consistent with an RMSE of about 1.5 px.
- **Ghosting:** moving objects produce ghosts. On the right, a motorcyclist who was present in only one of two overlapping shots appears semi-transparent in front of the white car, because feathering averages the two images. With hard-seam masks, multi-band blending replaces this half-transparent copy by an opaque one on our panorama (see below).
- **Perspective distortion:** projecting every image onto the plane of the third image strongly stretches the leftmost image into a large trapezoid (the cars and the tree on the left look much larger), and the canvas (5437×3331 for 1000×1500 inputs) has large black wedges. The stretching grows like $\tan\theta$ with the angle from the reference view; with the uncropped photos (about 140° in total) it diverges and no planar panorama can be built. A cylindrical or spherical projection, or photos taken with a narrower field of view, are needed for wide panoramas.
- **SIFT vs. ORB:** both give visually equivalent panoramas on this scene. SIFT

provides more matches and more inliers, which makes it more robust on difficult image pairs, at a much higher cost; ORB is far faster and, thanks to its many corner features on the facades, accurate enough here.

Blending: feathering vs. multi-band (bonus; Table 10, Figures 15 and 16**).**

Table 10: Bonus: blending methods on the SIFT panorama. Seam gradient: lower = less visible seam; overlap sharpness: mean $|\nabla^2 I|$ on the overlaps (higher = less blur and ghosting).

method	levels	seam gradient	overlap sharpness	sharpness vs hard seam (%)	time (ms)
Feathering	-	44.29	11.48	86.3	1963
Hard seam	0	72.69	13.30	100.0	2854
Multi-band	2	52.47	13.27	99.7	7182
	4	50.65	13.27	99.8	7103
	6	50.67	13.28	99.8	7049

- **Hard seam:** copying every pixel from a single image keeps the full sharpness but leaves visible steps: the seam gradient is the highest (72.7), the exposure change appears as a sharp line in the sky (Figure 16, bottom), and the seam cuts through the motorcyclist and the car (top).
- **Feathering** hides the seams best (seam gradient 44.3), but only because it also blurs everything along them: the overlap sharpness falls to 86.3% of the hard seam, and the motorcyclist becomes a semi-transparent ghost through which the car is visible.
- **Multi-band blending** combines both advantages: it removes most of the seam step (72.7 $\rightarrow$ 50.7 with 4 or 6 levels, a reduction of 30%) and the sky is smooth, while its overlap-sharpness index reaches 99.8% of that of the hard seam. On our panorama the motorcyclist is opaque and sharp instead of ghosted; this comes from the hard-seam masks, which take each detail from a single image, combined with the band-limited transition. Two levels already give most of the gain (52.5); beyond four levels the measures no longer change.
- **Limits and cost:** multi-band blending does not choose *where* the seam goes, so it does not remove ghosting in general: a moving object crossed by the seam can still be cut or blurred, as on the cap of the motorcyclist, where a short blurred band remains. These conclusions are limited to this set of photos. A graph-cut seam that avoids moving objects would solve this. It is also about 3.6 times slower than feathering (7.0 vs. 2.0 s), because it builds four floating-point pyramids and fills the area outside each image on the 18-megapixel canvas.

5 Conclusion

Key findings. (1) No filter is best for every noise type: the median filter is clearly superior for impulse noise, while a Gaussian filter with $\sigma \approx 0.5$–1 is best for Gaussian noise, the exact value depending on how much fine detail the image contains and on the metric (PSNR or SSIM); larger kernels trade edge sharpness for noise removal, and Laplacian sharpening amplifies noise and should only follow denoising with a small α. (2) RANSAC estimates the fundamental matrix of the stereo pair with sub-pixel Sampson

error. SGBM gives denser, smoother and more accurate disparity than BM at about four times the cost; `numDisparities` must cover the true disparity range, and with penalties P_1, P_2 proportional to b^2 the SGBM `blockSize` must stay small. (3) For stitching real street photos, SIFT finds twice as many matches as ORB, both reach about 1.5 px RMSE, and ORB is about nine times faster to detect. With parallax and moving objects the RANSAC threshold becomes a real trade-off ($t = 3$ px is a good compromise), feathering hides seams and exposure changes but leaves ghosts of moving objects, on our panorama, multi-band blending with hard-seam masks replaces these ghosts by opaque copies while still hiding the seams (overlap-sharpness index 99.8% of the hard seam, 30% lower seam step), although objects crossed by the seam can still be cut, and a planar projection only works for a limited field of view.

Future work. Edge-preserving filters (bilateral, non-local means) for Part A; calibrating a real camera and capturing our own stereo pair to obtain metric depth, with a left–right consistency check and hole filling, for Part B; a cylindrical projection to keep the full field of view of the photos, exposure compensation, and a graph-cut seam that avoids moving objects, combined with multi-band blending, for Part C.

6 References

[1] OpenCV Documentation, https://docs.opencv.org/4.x/.

[2] S. Baker, D. Scharstein, J. P. Lewis, S. Roth, M. J. Black, R. Szeliski, "A Database and Evaluation Methodology for Optical Flow," *IJCV*, 92(1):1–31, 2011. Data: https://vision.middlebury.edu/flow/data/.

[3] H. Hirschmüller, "Stereo Processing by Semiglobal Matching and Mutual Information," *IEEE TPAMI*, 30(2):328–341, 2008.

[4] R. Hartley, A. Zisserman, *Multiple View Geometry in Computer Vision*, 2nd ed., Cambridge University Press, 2004.

[5] D. G. Lowe, "Distinctive Image Features from Scale-Invariant Keypoints," *IJCV*, 60(2):91–110, 2004.

[6] E. Rublee, V. Rabaud, K. Konolige, G. Bradski, "ORB: An Efficient Alternative to SIFT or SURF," *ICCV*, 2011.

[7] P. J. Burt, E. H. Adelson, "A Multiresolution Spline with Application to Image Mosaics," *ACM Transactions on Graphics*, 2(4):217–236, 1983.

[8] Z. Wang, A. C. Bovik, H. R. Sheikh, E. P. Simoncelli, "Image Quality Assessment: From Error Visibility to Structural Similarity," *IEEE TIP*, 13(4):600–612, 2004.

Appendix A: Selected Code Listings

This appendix presents selected functions from the accompanying Jupyter notebook, `22070920_NguyenDinhKhai.ipynb`. The complete implementation, input data, experimental results, dependency list, and execution instructions are included in the submission package. To reproduce the experiments, open the notebook from the directory containing `data/`, select the appropriate Python environment, and run all cells in order. Execution

times may vary with the hardware and system load.

A.1 Part A: Image Filtering

```python
# Part A: selected functions from the accompanying notebook.

def add_gaussian_noise(img, sigma, rng):
    """Add zero-mean Gaussian noise in float, then clip to [0, 255]."""
    return np.clip(img + rng.normal(0, sigma, img.shape), 0, 255).astype(np.uint8)

def add_salt_pepper(img, density, rng):
    """Set each pixel to 0 or 255, each with probability density / 2."""
    out = img.copy()
    r = rng.random(img.shape[:2])
    out[r < density / 2] = 0
    out[r > 1 - density / 2] = 255
    return out

def laplacian_sharpen(img, k, alpha):
    """I_sharp = I - alpha * Laplacian(I). The k x k OpenCV kernel is divided by its
    response to f = x^2 (true Laplacian = 2), so alpha has the same meaning for every k."""
    x = np.arange(-10, 11, dtype=np.float32)
    gain = cv2.Laplacian(np.tile(x ** 2, (21, 1)), cv2.CV_32F, ksize=k)[10, 10] / 2.0
    f = img.astype(np.float32)
    lap = cv2.Laplacian(f, cv2.CV_32F, ksize=k) / gain
    return np.clip(f - alpha * lap, 0, 255).astype(np.uint8)

def filter_configs():
    """List of (filter name, parameters, function) for every tested setting."""
    cfgs = [("Mean", f"{k}x{k}", lambda im, k=k: cv2.blur(im, (k, k))) for k in KERNELS]
    cfgs += [("Gaussian", f"{k}x{k}, sigma={s}", lambda im, k=k, s=s: cv2.GaussianBlur(im, (k, k), s))
             for k in KERNELS for s in SIGMAS]
    cfgs += [("Median", f"{k}x{k}", lambda im, k=k: cv2.medianBlur(im, k)) for k in KERNELS]
    cfgs += [("Laplacian sharpen", f"{k}x{k}, alpha={a}", lambda im, k=k, a=a: laplacian_sharpen(im, k,
    ↪  a))
             for k in KERNELS for a in ALPHAS]
    return cfgs

def metrics(clean, img):
    """PSNR, SSIM (colour, channel_axis=2) and MSE against the clean image."""
    return {"PSNR (dB)": peak_signal_noise_ratio(clean, img, data_range=255),
            "SSIM": structural_similarity(clean, img, channel_axis=2, data_range=255),
            "MSE": mean_squared_error(clean, img)}
```

A.2 Part B: 3D Reconstruction

```python
# Part B: selected functions from the accompanying notebook.

def match_points(gl, gr):
    """SIFT + brute-force KNN (k=2) + Lowe ratio test; returns matched left/right points."""
    sift = cv2.SIFT_create()
    kp1, d1 = sift.detectAndCompute(gl, None)
    kp2, d2 = sift.detectAndCompute(gr, None)
    knn = cv2.BFMatcher(cv2.NORM_L2).knnMatch(d1, d2, k=2)
    good = [m for m, n in knn if m.distance < RATIO * n.distance]   # ratio test
    p1 = np.float32([kp1[m.queryIdx].pt for m in good])
    p2 = np.float32([kp2[m.trainIdx].pt for m in good])
    return p1, p2, len(kp1), len(kp2)

def epipolar_errors(F, p1, p2):
    """Symmetric point-to-epipolar-line distance and Sampson error (px) of each match."""
    x1 = np.c_[p1, np.ones(len(p1))]
    x2 = np.c_[p2, np.ones(len(p2))]
    l2 = x1 @ F.T                                   # epipolar lines in the right image: F x1
    l1 = x2 @ F                                     # epipolar lines in the left image: F^T x2
    num = np.sum(x2 * l2, axis=1)                   # x2^T F x1 (0 for a perfect match)
    d_epi = 0.5 * (np.abs(num) / np.hypot(l2[:, 0], l2[:, 1]) + np.abs(num) / np.hypot(l1[:, 0], l1[:,
    ↪  1]))
    sampson = np.abs(num) / np.sqrt(l2[:, 0] ** 2 + l2[:, 1] ** 2 + l1[:, 0] ** 2 + l1[:, 1] ** 2)
```

```
    return d_epi, sampson

def compute_disparity(method, nd, bs, left, right):
    """Disparity (px) by StereoBM (grey) or StereoSGBM (colour); invalid pixels = NaN."""
    if method == "BM":
        m = cv2.StereoBM_create(numDisparities=nd, blockSize=bs)
        m.setMinDisparity(MIN_DISP)
        m.setUniquenessRatio(10)
        m.setSpeckleWindowSize(100)
        m.setSpeckleRange(2)
        left, right = cv2.cvtColor(left, cv2.COLOR_BGR2GRAY), cv2.cvtColor(right, cv2.COLOR_BGR2GRAY)
    else:
        # P1: penalty for 1 px disparity changes, P2: for larger jumps (OpenCV guideline)
        m = cv2.StereoSGBM_create(minDisparity=MIN_DISP, numDisparities=nd, blockSize=bs,
                                  P1=8 * 3 * bs * bs, P2=32 * 3 * bs * bs, disp12MaxDiff=1,
                                  uniquenessRatio=10, speckleWindowSize=100, speckleRange=2)
    d = m.compute(left, right).astype(np.float32) / 16.0       # OpenCV returns disparity x 16
    return np.where(d >= MIN_DISP, d, np.nan)

def disparity_metrics(disp, gt):
    """Valid ratio, RMSE and bad-pixel rates against the ground truth."""
    valid = np.isfinite(disp)
    m = valid & np.isfinite(gt)
    err = np.abs(disp[m] - gt[m])
    has = err.size > 0                                          # some settings have no valid pixel
    return {"valid ratio (%)": 100 * valid.mean(),
            "RMSE (px)": np.sqrt(np.mean(err ** 2)) if has else np.nan,
            "bad>1 (%)": 100 * np.mean(err > 1) if has else np.nan,
            "bad>2 (%)": 100 * np.mean(err > 2) if has else np.nan,
            # invalid pixels count as errors
            "bad>2 incl. invalid (%)": 100 * (1 - np.mean((np.abs(disp - gt) <= 2)[np.isfinite(gt)]))}

def point_cloud(disp, left, out_dir):
    """Reproject the disparity to 3D with Q, save a .ply file and four views."""
    h, w = disp.shape
    # [X Y Z W] = Q [x y d 1]  =>  Z = f B / (d + offset)
    Q = np.float32([[1, 0, 0, -w / 2], [0, 1, 0, -h / 2], [0, 0, 0, FOCAL_PX],
                    [0, 0, 1 / BASELINE, DISP_OFFSET / BASELINE]])
    pts = cv2.reprojectImageTo3D(np.nan_to_num(disp, nan=-100).astype(np.float32), Q)
    ok = np.isfinite(disp) & (disp + DISP_OFFSET > 0.5)            # keep finite, positive depths
    xyz, rgb = pts[ok], cv2.cvtColor(left, cv2.COLOR_BGR2RGB)[ok]
    lo, hi = np.percentile(xyz[:, 2], [1, 99])                     # drop 1% outliers at each end
    keep = (xyz[:, 2] > lo) & (xyz[:, 2] < hi)
    xyz, rgb = xyz[keep], rgb[keep]
    header = ("ply\nformat ascii 1.0\nelement vertex %d\nproperty float x\nproperty float y\nproperty
    ↪  float z\n"
              "property uchar red\nproperty uchar green\nproperty uchar blue\nend_header" % len(xyz))
    np.savetxt(out_dir / "pointcloud_sgbm.ply", np.c_[xyz, rgb], fmt="%.4f %.4f %.4f %d %d %d",
               header=header, comments="")
    # plot (X, Z, -Y): depth on the second axis
    sub = np.random.default_rng(0).choice(len(xyz), min(30000, len(xyz)), replace=False)
    views = [("front", 0, -90), ("left", 10, -60), ("right", 10, -120), ("top", 70, -90)]
    fig = plt.figure(figsize=(10, 9))
    for i, (name, elev, azim) in enumerate(views, start=1):
        ax = fig.add_subplot(2, 2, i, projection="3d")
        ax.scatter(xyz[sub, 0], xyz[sub, 2], -xyz[sub, 1], c=rgb[sub] / 255.0, s=0.8)
        ax.view_init(elev=elev, azim=azim)
        ax.set_box_aspect(np.ptp(xyz[sub][:, [0, 2, 1]], axis=0), zoom=1.4)   # less white space
        ax.set_title(name)
        ax.set_axis_off()
    plt.tight_layout()
    plt.savefig(out_dir / "pointcloud_views.png", dpi=150)
    plt.close(fig)
    return len(xyz)
```

A.3 Part C: Image Stitching

```
# Part C: selected functions from the accompanying notebook.

def match_pair(des_a, des_b, norm):
    """KNN (k=2) matches from image a to image b that pass Lowe's ratio test."""
```

```
    if des_a is None or des_b is None or len(des_b) < 2:
        return []
    knn = cv2.BFMatcher(norm).knnMatch(des_a, des_b, k=2)
    return [p[0] for p in knn if len(p) == 2 and p[0].distance < RATIO * p[1].distance]

def find_homography(src, dst, thresh):
    """RANSAC homography src -> dst; returns H, the inlier mask and the inlier RMSE."""
    if len(src) < 4:
        return None, np.zeros(len(src), bool), np.nan
    cv2.setRNGSeed(42)                                          # fixed seed for reproducibility
    H, mask = cv2.findHomography(src, dst, cv2.RANSAC, thresh)
    if H is None:
        return None, np.zeros(len(src), bool), np.nan
    inl = mask.ravel().astype(bool)
    proj = cv2.perspectiveTransform(src[inl].reshape(-1, 1, 2), H).reshape(-1, 2)
    rmse = np.sqrt(np.mean(np.sum((proj - dst[inl]) ** 2, axis=1)))   # sqrt(mean ||x' - H x||^2)
    return H, inl, rmse

def warp_images(images, pair_H):
    """Warp all images into the frame of the middle image on a common canvas.
    pair_H[i] maps image i+1 -> i (chained to the right, inverted to the left)."""
    n, ref = len(images), len(images) // 2
    H = [None] * n
    H[ref] = np.eye(3)
    for i in range(ref + 1, n):
        H[i] = H[i - 1] @ pair_H[i - 1]                         # i -> i-1 -> ... -> ref
    for i in range(ref - 1, -1, -1):
        H[i] = H[i + 1] @ np.linalg.inv(pair_H[i])              # i -> i+1 -> ... -> ref
    corners = []
    for im, Hi in zip(images, H):
        h, w = im.shape[:2]
        c = np.float32([[0, 0], [w, 0], [w, h], [0, h]]).reshape(-1, 1, 2)
        corners.append(cv2.perspectiveTransform(c, Hi).reshape(-1, 2))
    corners = np.vstack(corners)
    x0, y0 = np.floor(corners.min(axis=0))
    x1, y1 = np.ceil(corners.max(axis=0))
    size = (int(x1 - x0), int(y1 - y0))
    if size[0] * size[1] > MAX_CANVAS_PIXELS:
        raise ValueError(f"canvas {size[0]}x{size[1]} px is too large (total rotation too wide for a
        ↪  planar panorama)")
    T = np.array([[1, 0, -x0], [0, 1, -y0], [0, 0, 1]])
    warped = [cv2.warpPerspective(im, T @ Hi, size) for im, Hi in zip(images, H)]
    # valid-pixel masks, eroded by 1 px to drop border pixels mixed with black
    masks = [cv2.erode(cv2.warpPerspective(np.full(im.shape[:2], 255, np.uint8), T @ Hi, size,
                                           flags=cv2.INTER_NEAREST), np.ones((3, 3), np.uint8))
             for im, Hi in zip(images, H)]
    return warped, masks

def blend_feather(warped, masks):
    """Feathering: weighted mean, weight = distance to the image border."""
    acc = np.zeros(warped[0].shape, np.float64)
    wsum = np.zeros(warped[0].shape[:2], np.float64)
    for im, m in zip(warped, masks):
        w = cv2.distanceTransform(m, cv2.DIST_L2, 5)
        acc += im * w[..., None]
        wsum += w
    return (acc / np.maximum(wsum, 1e-6)[..., None]).astype(np.uint8)

def seam_masks(masks):
    """Hard seam: each pixel belongs to the image with the largest distance to its border."""
    dist = np.stack([cv2.distanceTransform(m, cv2.DIST_L2, 5) for m in masks])
    owner, covered = dist.argmax(axis=0), dist.max(axis=0) > 0
    return [((owner == k) & covered).astype(np.float32) for k in range(len(masks))]

def fill_outside(im, mask):
    """Fill pixels outside the image with the nearest valid colour (avoids dark borders)."""
    _, labels = cv2.distanceTransformWithLabels((mask == 0).astype(np.uint8), cv2.DIST_L2, 5,
                                                labelType=cv2.DIST_LABEL_PIXEL)
    src = np.flatnonzero(mask.ravel())                          # label k (from 1) = k-th valid pixel in
    ↪  raster order
    return im.reshape(-1, 3)[src[labels.ravel() - 1]].reshape(im.shape)

def blend_multiband(warped, masks, levels):
```

```
    """Multi-band blending: each Laplacian level is blended with the seam mask blurred
    to the same level, then the pyramid is collapsed. levels = 0 gives the hard seam."""
    acc = wsum = None
    for im, m, s in zip(warped, masks, seam_masks(masks)):
        src = fill_outside(im, m) if levels else im            # no filling needed for the hard seam
        g = gaussian_pyramid(src.astype(np.float32), levels)
        lap = [g[l] - cv2.pyrUp(g[l + 1], dstsize=g[l].shape[1::-1]) for l in range(levels)] + [g[-1]]
        w = gaussian_pyramid(s, levels)
        if acc is None:
            acc, wsum = [np.zeros_like(x) for x in lap], [np.zeros_like(x) for x in w]
        for l in range(levels + 1):
            acc[l] += lap[l] * w[l][..., None]
            wsum[l] += w[l]
    # normalise: near the canvas border the blurred masks sum to less than 1
    out = acc[-1] / np.maximum(wsum[-1], 1e-6)[..., None]
    for l in range(levels - 1, -1, -1):
        out = cv2.pyrUp(out, dstsize=acc[l].shape[1::-1]) + acc[l] / np.maximum(wsum[l], 1e-6)[..., None]
    covered = np.any(np.stack(masks) > 0, axis=0)
    return np.where(covered[..., None], np.clip(out, 0, 255), 0).astype(np.uint8)
```